REALITYOS

AI-Powered Decision Simulation Website

Simulate the decision before you live it.

Project report · Product design, system analysis and implementation

ABSTRACT

RealityOS is a website that helps people reason through consequential decisions before acting. A user describes a situation in plain language—including goals, deadlines, available time, constraints and resources. The website transforms that description into a structured model, generates distinct decision strategies, estimates their outcomes, surfaces risks and explains a recommendation.

Unlike a conventional chatbot, RealityOS closes the loop: users can later record what actually happened, compare reality with the original prediction and use their own history to calibrate future estimates. A deterministic simulation engine and trained workload models provide the numerical estimates; optional language-model agents help with interpretation and explanations. When no language-model service is configured or available, the website continues to work using its local reasoning core.

The product is implemented as a responsive React website, a same-origin API on a Cloudflare Worker, and a D1 database. Authentication, private user data, input validation, ownership checks and account deletion are part of the implementation.

PROJECT PURPOSE

Problem

Important choices often involve competing goals, uncertain workloads, limited time and dependencies. People commonly compare options informally, underestimate effort or fail to revisit how an earlier decision turned out. Advice that sounds confident is not necessarily grounded in a reproducible estimate.

Goal

Provide a clear, repeatable decision workflow that helps users:

- describe a real situation in their own words;
- see the situation organized into goals, deliverables, constraints and risks;
- compare several materially different paths;
- understand feasibility, workload, deadline risk and uncertainty;
- test changes with What-If scenarios;
- record the real outcome and learn from prediction errors.

Product principle

The system separates **numbers** from **narrative**. The simulation engine owns the probabilities and workload projections. AI agents may propose strategies or explain trade-offs, but their numeric claims are not trusted as model output. Estimates are presented as estimates, never guarantees.

SYSTEM ANALYSIS — COMMON APPROACHES

Decision-making is often handled with informal notes, spreadsheets, general-purpose chatbots or task lists. These approaches can be useful, but they typically require users to create their own comparison structure and may not connect an original forecast with the eventual outcome.

Limitations addressed

- Comparisons can omit important constraints or alternatives.
- Workload and deadline assumptions are difficult to inspect.
- A persuasive answer may not reveal how its numbers were produced.
- The original prediction is often not retained for later evaluation.
- Advice is not necessarily personalized using a user's recorded outcomes.

RealityOS addresses these gaps with a structured simulation workflow, explainable factors, explicit feasibility signals and a prediction-versus-reality feedback loop. It does not claim to replace professional advice or guarantee a successful result.

PROPOSED WEBSITE

A user enters a situation once and follows it through a complete decision lifecycle:

1. **Capture** — describe the goal, context, time available, deadlines and constraints; optionally add voice input or attach a document for in-browser text extraction.
2. **Structure** — extract and edit the situation model, including deliverables, dependencies, resources and open questions.
3. **Plan** — generate three or four distinct strategies with assumptions and trade-offs.
4. **Simulate** — estimate required and available hours, coverage, expected completion, completion likelihood, deadline risk and workload pressure.
5. **Challenge** — inspect option-specific risks and assumptions rather than viewing a recommendation in isolation.
6. **Compare and choose** — review a transparent ranking and the reasons the recommended option scores well.
7. **Record reality** — enter what happened when the decision has played out.
8. **Learn** — compare forecast with outcome and apply cautious, per-user calibration to later simulations.

The public website explains this workflow and includes an interactive demonstration driven by the actual in-browser simulation engine.

WEBSITE FEATURES

Public pages

Page	Purpose
Home	Product introduction and live What-If demonstration
How it works	Explains the decision-simulation loop
Features	Product capabilities and model information
About	Product philosophy and documentation
Privacy	Data handling and account deletion information
Login and sign up	Sign-in entry through Freebuff authentication

Page	Purpose
Account recovery	Guidance for identity-provider account access
Not found	Friendly handling for unknown routes

Signed-in workspace

Area	Functionality
Dashboard	Recent decisions, counts, learning snapshot and next steps
New simulation	Text, optional voice, file attachments and structured hints
Simulation history	Browse saved decisions and their status
Simulation detail	Situation analysis, strategy comparison, recommendation and outcome flow
What-If	Change selected assumptions and inspect projected metric deltas
Decision profile	Personal reliability, calibration, accuracy and trends
Settings	Theme, model information, training export and account deletion
Onboarding	Capture preferences and create a clearly labelled demo simulation

Accessible design and recovery

The website uses a responsive React interface, light and dark themes, shared components, loading and empty states, and an error recovery boundary intended to prevent a rendering failure from leaving the entire interface blank. Ambient background decoration is non-interactive and designed to respect reduced-motion preferences. These interface safeguards improve resilience but cannot prevent an outage at the hosting provider.

TECHNICAL REQUIREMENTS

Software

- Browser-based website built with React 19, TypeScript and Vite.
- Client-side routing with lazy-loaded pages.
- Tailwind CSS v4, Motion animations and Recharts visualizations.
- Cloudflare Worker using Hono for the API and static website delivery.
- Cloudflare D1 (SQLite) for persistent data.
- Freebuff OpenID Connect authentication with PKCE and HTTP-only session cookies.
- Zod schemas for runtime validation.
- Optional OpenAI-compatible language-model endpoint; deterministic local engine works without it.
- Node.js 20 or newer for development, training, tests and build tooling.

Hosting architecture

The repository includes Cloudflare Worker configuration and deployment documentation. A deployment requires an authorized hosting account, correctly provisioned D1 binding and registered authentication callback for the final public origin. A generated configuration is not itself proof that deployment has occurred or that a provider URL is available.

SYSTEM ARCHITECTURE

```
Browser
  React website · client routing · charts · forms · local demo engine
       | same-origin HTTP requests and server-sent events
       ▼
Cloudflare Worker
  Freebuff auth wrapper → Hono API → services → AI agents / local engine
       |                                      |
       |                                      └─ shared simulation + ML
       ▼
D1 / SQLite
  users · profiles · simulations · situations · runs · predictions
  outcomes · comparisons · training rows · rate limits · agent events

Worker static assets binding serves the built website.
```

The design has three deliberate boundaries:

- **Deterministic core:** pure workload, capacity, feature extraction, model inference, simulation, scoring and What-If calculations, shared between browser and Worker.
- **Language layer:** schema-validated agents that propose strategies, risks and explanations; invalid or unavailable responses fall back to the local core.
- **Platform layer:** HTTP routing, authentication, database persistence, rate limiting and authorization.

IMPLEMENTATION MODULES

Situation intake and document support

The user can describe a decision and optionally provide attachments. PDF, DOCX and plain-text extraction occurs in the browser; heavy parsers are loaded lazily. Extracted text can support the situation workflow without a file-storage service or uploading original document files for storage.

Simulation and workload model

A shared deterministic engine estimates workload and capacity, simulates options and exposes the factors contributing to results. The shipped model includes two logistic regressions (completion and deadline miss) and a ridge regressor (workload pressure). The bundled model card identifies its version, data source and evaluation metrics.

The current model was trained offline on a seeded, simulated cohort of 12,000 samples—not on a large collection of real user outcomes. The documentation reports completion AUC 0.865, deadline-miss AUC 0.872 and stress-model R^2 0.708 for that generated evaluation. Those figures describe the documented cohort experiment; they do not guarantee accuracy for an individual or for real-world populations.

AI agent pipeline

The optional pipeline separates situation understanding, planning, simulation, risk analysis, criticism, recommendation and reflection. Agent output is validated against schemas and normalized before use. The engine—not the language model—computes stored simulation probabilities. If no provider is configured or an agent fails, the deterministic local reasoning core provides a working fallback.

Learning from outcomes

Each run stores its prediction and the relevant feature data. When a user records an outcome, the system computes the gap, produces a reflection and adds a training row. Shrunk per-user reliability and bias correction feed later simulations gradually, so one unusual result cannot dominate the calibration. Users can export training rows for offline retraining.

DATA, AUTHENTICATION AND SECURITY

Data model

D1 tables store identity mirrors and profiles, simulations, structured situations, run results, predictions, outcomes, comparisons, training examples, rate limits and agent events. The signed-in user's stable identity (`sub`) is the ownership key for private records.

Sign-in

Freebuff handles identity using OpenID Connect and PKCE. RealityOS does not store passwords. The Worker uses a canonical configured application origin, and the authentication callback must be registered for the exact deployed hostname at `/auth/freebuff/callback`.

Safeguards implemented

- Every private query is scoped to the signed-in user's identity.
- State-changing API requests are checked for same-origin origin.
- Request bodies and model output are validated at boundaries.
- Mutating endpoints use per-user rate limits.
- Private API responses are marked `no-store`.
- LLM credentials remain server-side and are not delivered to the browser.
- Account deletion removes the user's profile and related decision records.
- Structured logging avoids recording tokens and cookies.

USER-FACING WORKFLOW

```
Describe a decision
        ↓
Review and edit the structured situation
        ↓
Analyze → compare distinct strategies → inspect risks
        ↓
Use What-If to test assumptions
        ↓
Choose a path and act in the real world
        ↓
Record the outcome → compare forecast with reality
        ↓
Personal calibration informs the next simulation
```

The website supports both the public learning journey and the signed-in workflow. The landing-page demo can run the real deterministic engine locally in the browser without requiring an account or backend round trip.

VERIFICATION AND TESTING

The repository documents three layers of automated checks:

- **Engine tests:** date/hour/deliverable extraction, determinism, monotonicity, scoring, What-If direction and prediction-versus-outcome calculations.
- **Agent tests:** valid and invalid provider output, hallucinated deliverables, fallback behaviour, honest engine reporting and stage events.
- **API end-to-end tests:** the Worker in real workerd with local D1, including unauthenticated responses, cross-origin rejection, account isolation, validation and the complete simulation lifecycle.

Standard project commands are `npm run typecheck`, `npm run build`, `npm test` and `node --test tests/api.test.mjs`. Automated checks support implementation confidence; they do not certify external hosting availability or complete an interactive Google sign-in in a user's browser.

LIMITATIONS AND FUTURE DEVELOPMENT

Known limitations

- Model training data is synthetic; broader validation with consented, representative real outcomes is needed before making strong performance claims.
- Predictions are estimates and depend on the quality of user inputs and model assumptions.
- Optional AI explanations require a configured provider and may not be available in every deployment; the local engine is the functional fallback.
- Sign-in depends on correct callback registration and availability of the identity and hosting providers.
- A website hosted on a provider that pauses or becomes unavailable will be inaccessible regardless of frontend error handling.

Future ideas

- Expand consent-based real-world evaluation and retraining with careful privacy and bias review.
- Add optional reminders for revisiting decisions and recording outcomes.
- Offer richer personal coaching and decision-pattern insights while keeping model uncertainty visible.
- Improve collaboration for decisions involving multiple stakeholders.
- Support exportable decision briefs and additional document formats.
- Add integrations for calendars and project tools only with explicit user authorization.

CONCLUSION

RealityOS is a fully implemented decision-simulation **website** centered on an inspectable, repeatable decision loop. Its distinctive capability is not simply generating advice: it structures a user's situation, compares different strategies using a shared deterministic engine, explains risks, supports What-If exploration, and records outcomes so predictions can be evaluated and cautiously calibrated over time.

The implementation combines a public product website and signed-in workspace with a React frontend, Hono Worker API, D1 persistence, Freebuff sign-in, optional AI agents and a deterministic fallback. Its boundaries and limitations are stated explicitly: the model is trained on a simulated cohort, outcomes are estimates, and deployment reliability depends on the selected hosting provider.

THANK YOU

RealityOS — Simulate the decision before you live it.